

## Q-1: Counting Pairs with Sum less than K

You are given a unique sorted array of integers  $A$  of size  $n$  and an integer  $K$ . Your task is to count the number of pairs  $(i, j)$  such that  $i < j$  and  $A[i] + A[j] < K$ .

### Constraints:

- The array is sorted in ascending order.
- Each element in the array is unique.
- **Expected time complexity:**  $\Theta(n \log n)$ .

### Example Test Cases:

**TEST CASE 1**

Input:  $n = 4$ ,  $K = 10$

$A = [1, 4, 6, 8]$

Output: 3

Explanation: The valid pairs are (1, 4), (1, 6), and (1, 8).

**TEST CASE 2**

Input:  $n = 5$ ,  $K = 12$

$A = [2, 3, 5, 7, 11]$

Output: 5

Explanation: The valid pairs are (2, 3), (2, 5), (2, 7), (3, 5), and (3, 7).

### Input Format:

- The first line contains two integers  $n$  (size of the array) and  $K$ .
- The second line contains  $n$  integers, the elements of the sorted array  $A$ .

### Output Format:

- Return the count of such pairs. (Use long long if exceeding)

(After reading this question, follow the steps described in the `Instructions.pdf` file to see the exact input and output format for the provided test cases.)

## Q-2: Maximum Running Time of N Computers

You have  $n$  computers. You are given the integer  $n$  and a 0-indexed sorted integer array `batteries` of length  $m$  ( $m \geq n$ ) where the  $i$ th battery can run a computer for `batteries[i]` minutes. You are interested in running all  $n$  computers simultaneously using the given batteries.

Initially, you can insert at most one battery into each computer. After that, and at any integer time moment, you can remove a battery from a computer and insert another battery any number of times. The inserted battery can be a totally new battery or a battery from another computer. You may assume that the removing and inserting processes take no time.

Note that the batteries cannot be recharged and the array is sorted.

Return the maximum number of minutes you can run all the  $n$  computers simultaneously.

### Example Test Cases:

**TEST CASE 1**

Input:  $n = 2, m = 3$

`batteries = [3,3,3]`

Output: 4

Explanation: Initially, insert battery 0 into the first computer and battery 1 into the second computer. After two minutes, remove battery 1 from the second computer and insert battery 2 instead. Note that battery 1 can still run for one minute. At the end of the third minute, battery 0 is drained, and you need to remove it from the first computer and insert battery 1 instead. By the end of the fourth minute, battery 1 is also drained, and the first computer is no longer running. We can run the two computers simultaneously for at most 4 minutes, so we return 4.

**TEST CASE 2**

Input:  $n = 3, m = 4$

`batteries = [4,5,6,9]`

Output: 7

**TEST CASE 3**Input:  $n = 2$ ,  $m = 4$ `batteries = [3,5,10,10]`

Output: 14

(Sum of batteries may exceed 32 bit integer so use a 64 bit variable like long long)

**Hint:**

Would you rather use the first  $n$  batteries or the last  $n$  batteries?

**Expected time complexity:**  $\Theta(m \log k)$  where  $k$  is the maximum power of one battery.

**Input Format:**

- The first line contains two integers  $n$  (number of computers) and  $m$  (size of batteries array).
- The second line contains  $m$  integers, the elements of the sorted array *batteries*.

**Output Format:**

- Return the maximum number of minutes you can run all the  $n$  computers simultaneously.

**Note:** In Q1, for  $O(n^2)$  only 2 ET's will pass, for Q2 only the optimal solution will pass the ET's.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)